

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

CON308

Taking ownership of public images with Amazon ECR

Matt Kang

Sr SDE, Amazon ECR
AWS

Adam Long

SDE, Amazon ECR
AWS



A common practice

```
FROM nginx:1.25.3
```

```
RUN apt-get update && \
    apt-get upgrade -y
```

```
COPY index.html \
    /usr/share/nginx/html
```

```
EXPOSE 8080
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: nginx
```

```
spec:
```

```
  containers:
```

```
    - name: nginx
```

```
      image: nginx:1.25.3
```

```
      ports:
```

```
        - containerPort: 80
```

```
..
```

A common practice

```
FROM nginx:1.25.3
```

```
RUN apt-get update && \
    apt-get upgrade -y
```

```
COPY index.html \
    /usr/share/nginx/html
```

```
EXPOSE 8080
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: nginx
```

```
spec:
```

```
  containers:
```

```
  - name: nginx
```

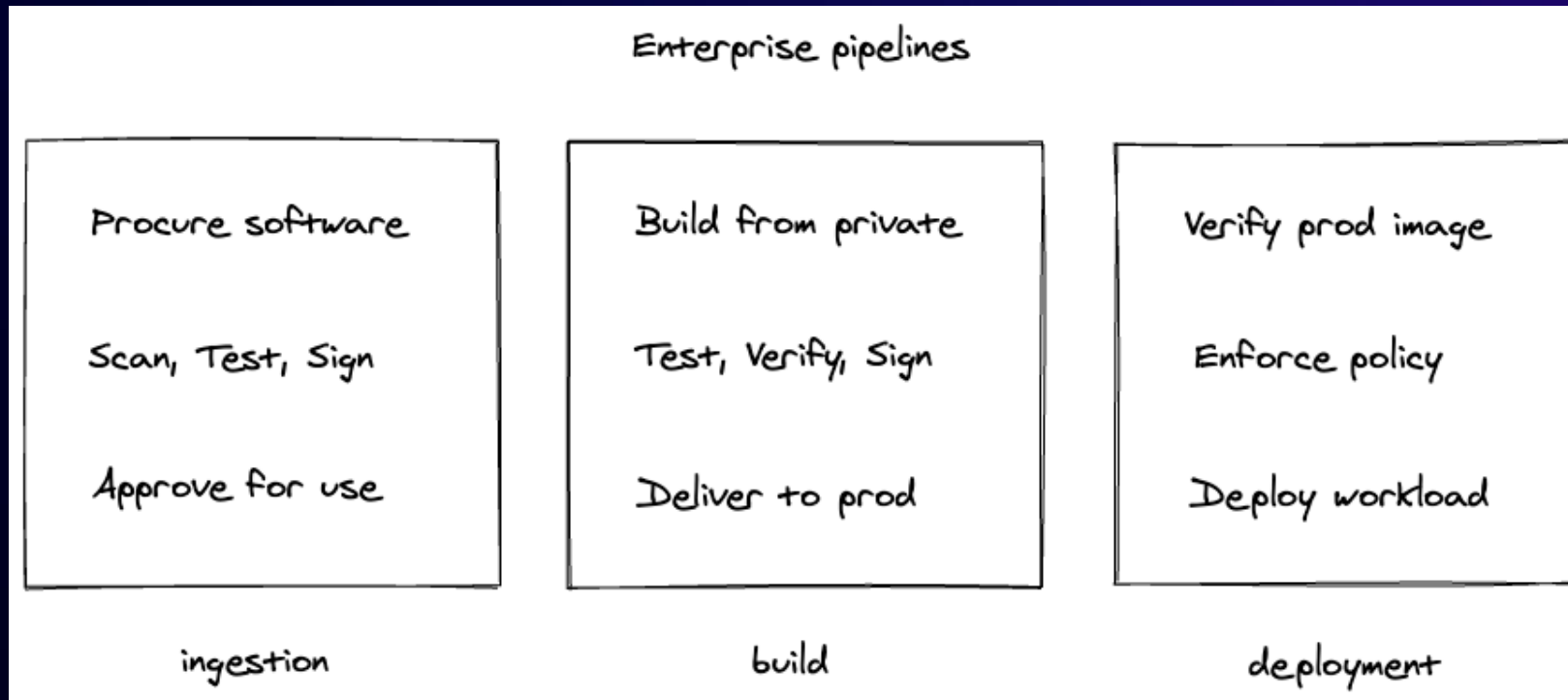
```
    image: nginx:1.25.3
```

```
    ports:
```

```
    - containerPort: 80
```

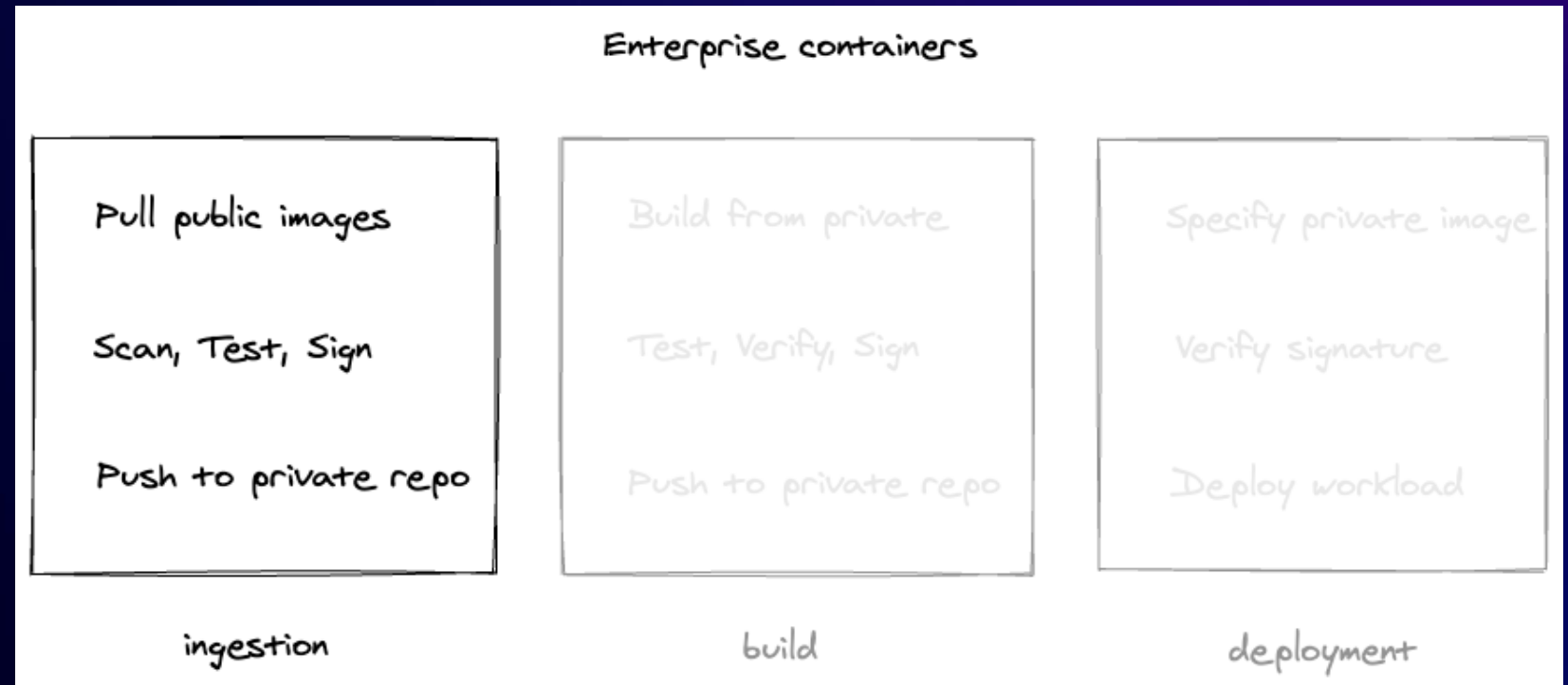
```
..
```

Enterprise software practice



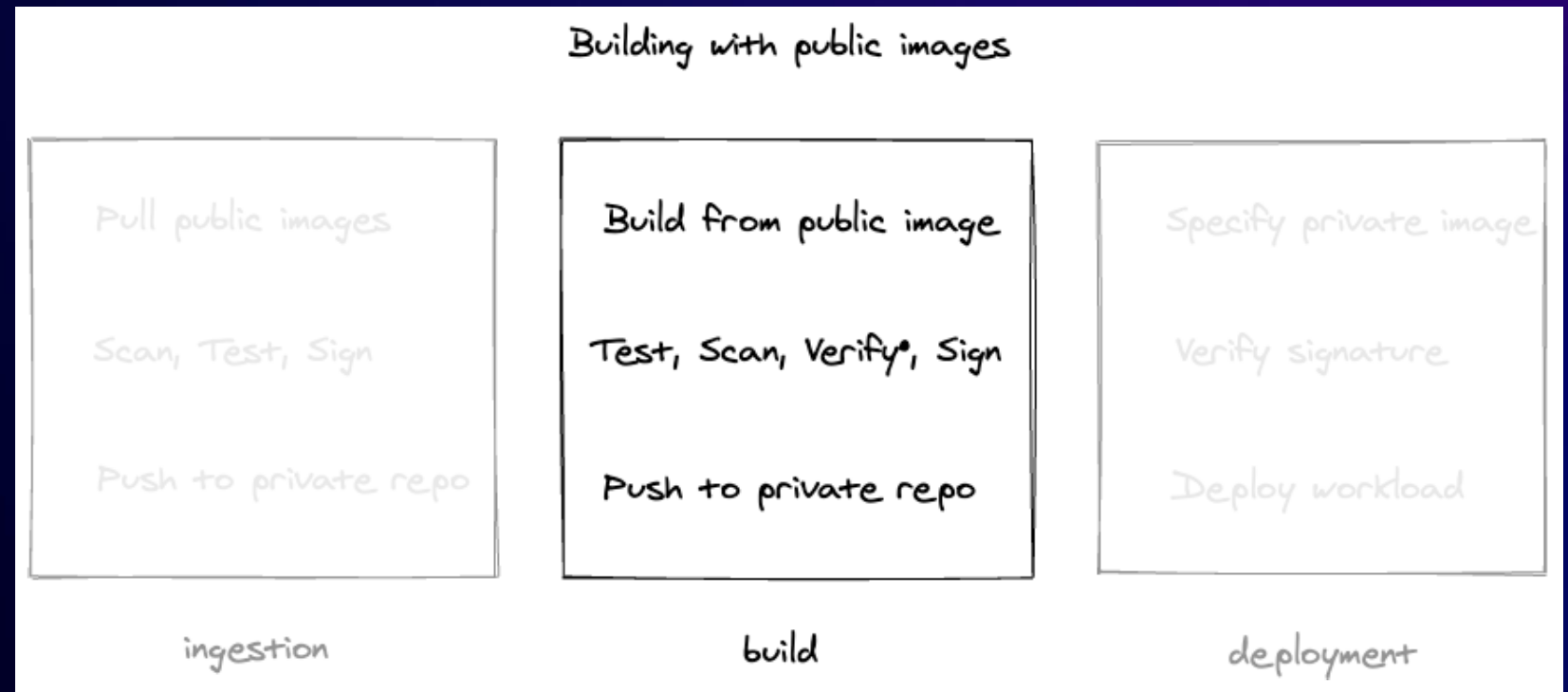
Enterprise containers in an ideal world

- Public images are vetted, scanned, tested, and signed
- Subsequent use is approved via image verification



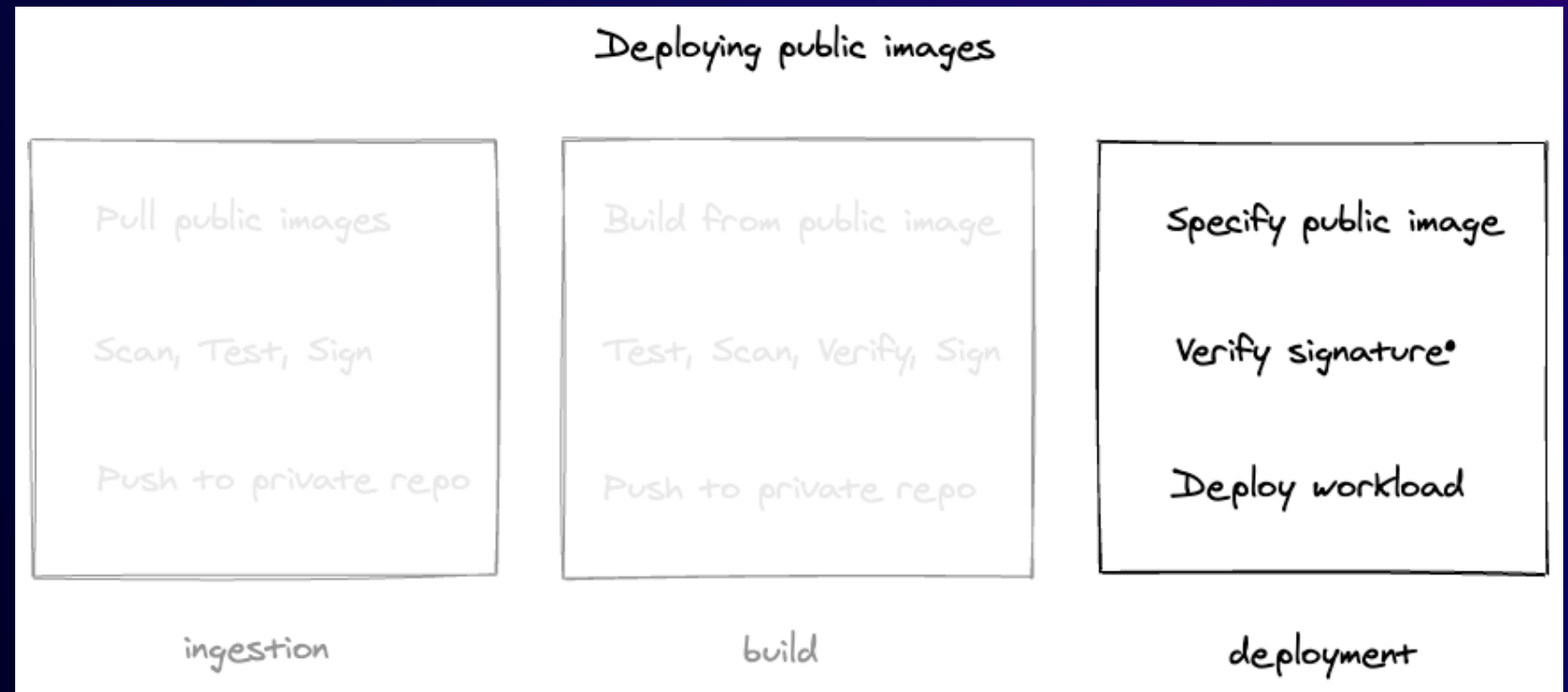
Building with public images

- Commonly, public images are trusted in a build pipeline
- Images may be verified, then scanned and signed
- Production use still gated



Deploying public images into production

- Very common pattern
- No ingestion controls
- Third-party scan and attestation is the only gate on production



Is an anti-pattern the most common pattern?

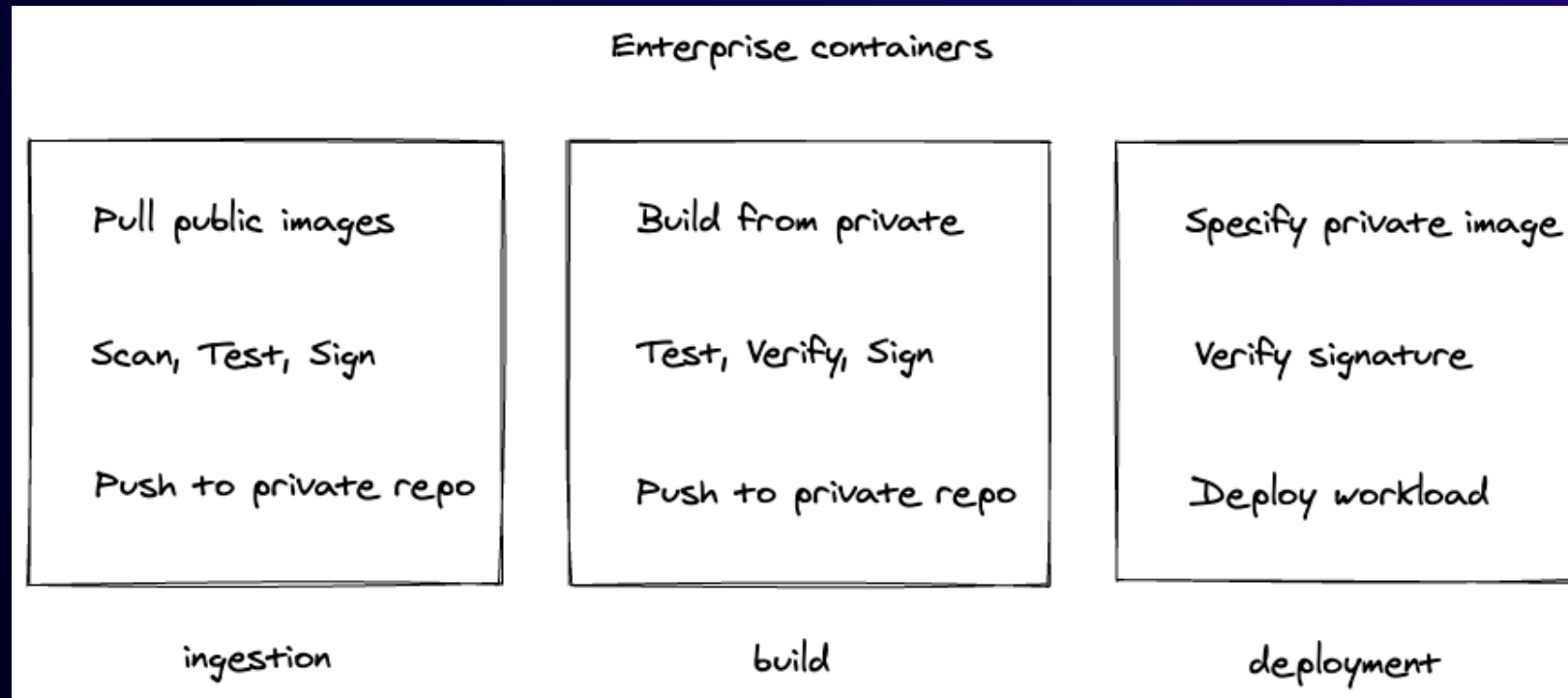
Developer velocity is a critical requirement

Typically, we don't fix what isn't broken

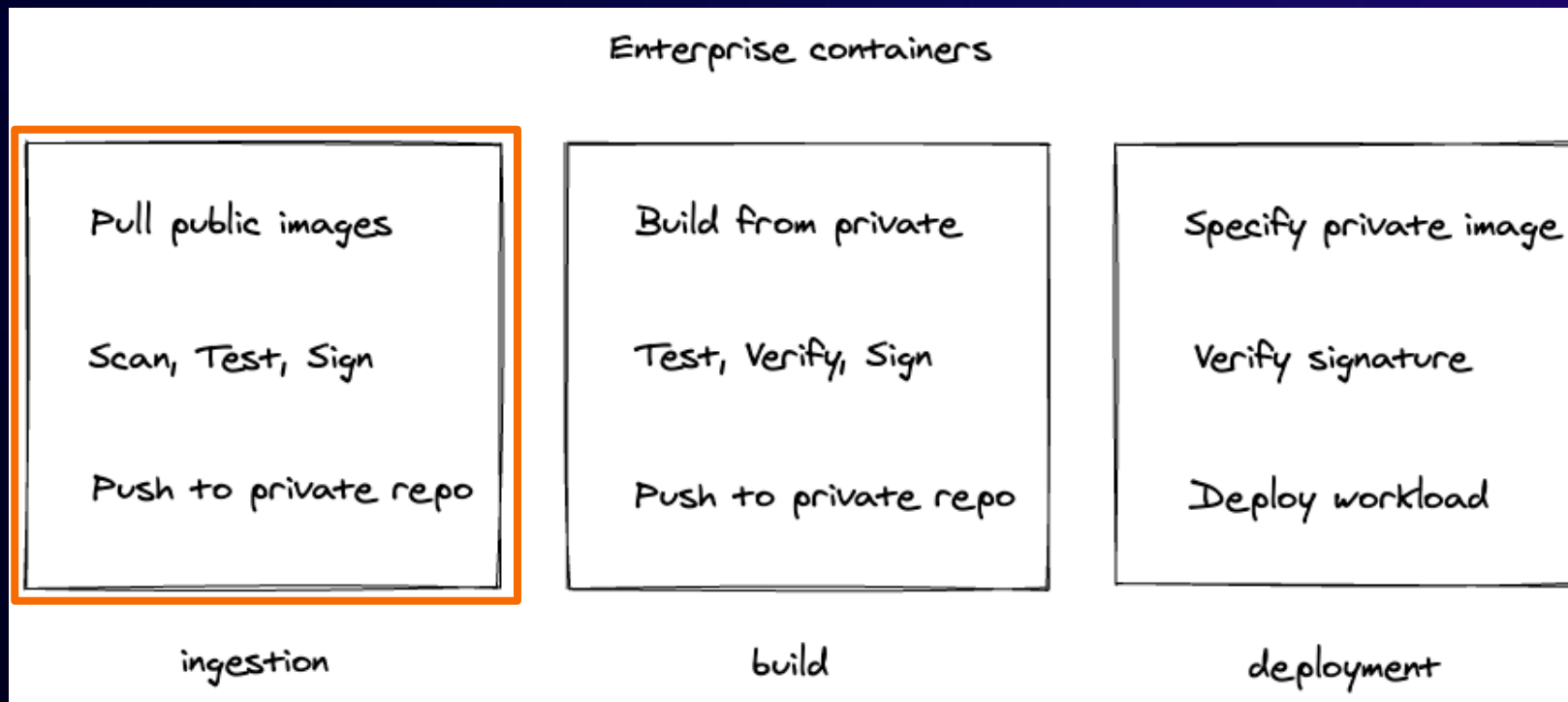
But there is an opportunity to make deployments safer, centralize governance, and enforce consistent policy

Business and engineering have a shared goal to address software supply chain security . . . if not now, when?

Can we simplify the ideal?

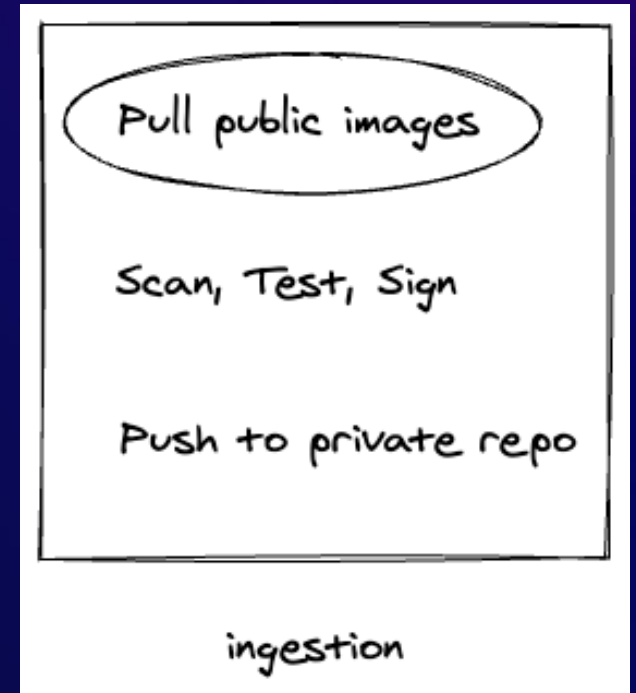


Can we simplify the ideal?



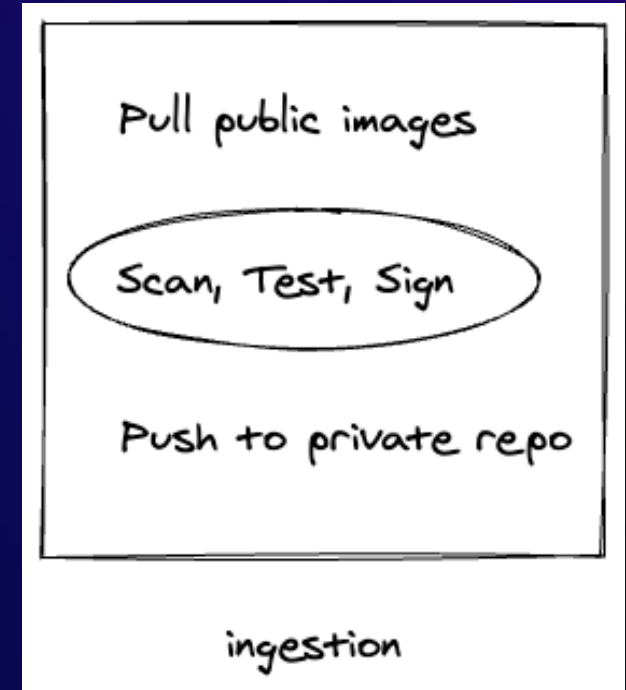
Ingestion pipelines: Procurement

- Establish trusted public registries
- Mirror images from upstream repos
- Verify images in quarantine



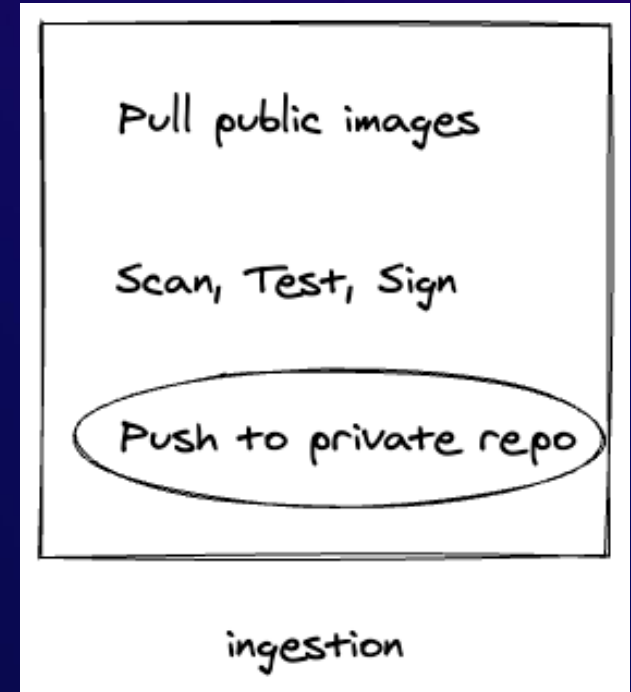
Ingestion pipelines: Quarantine

- Quarantine and limit access to new images
- Scan for vulnerabilities, respond to findings
- Approve images and sign them as trusted



Ingestion pipelines: Promote and distribute

- Migrate across the dev/prod boundary
- Images can now be deployed in workloads or used in pipelines



What does this mean in practice?

Ok ECR, do it for me!

In reality, solutions are nuanced,
but the building blocks are there

Let's walk through an opinionated
example that you can make use of



Ingestion solution: ECR

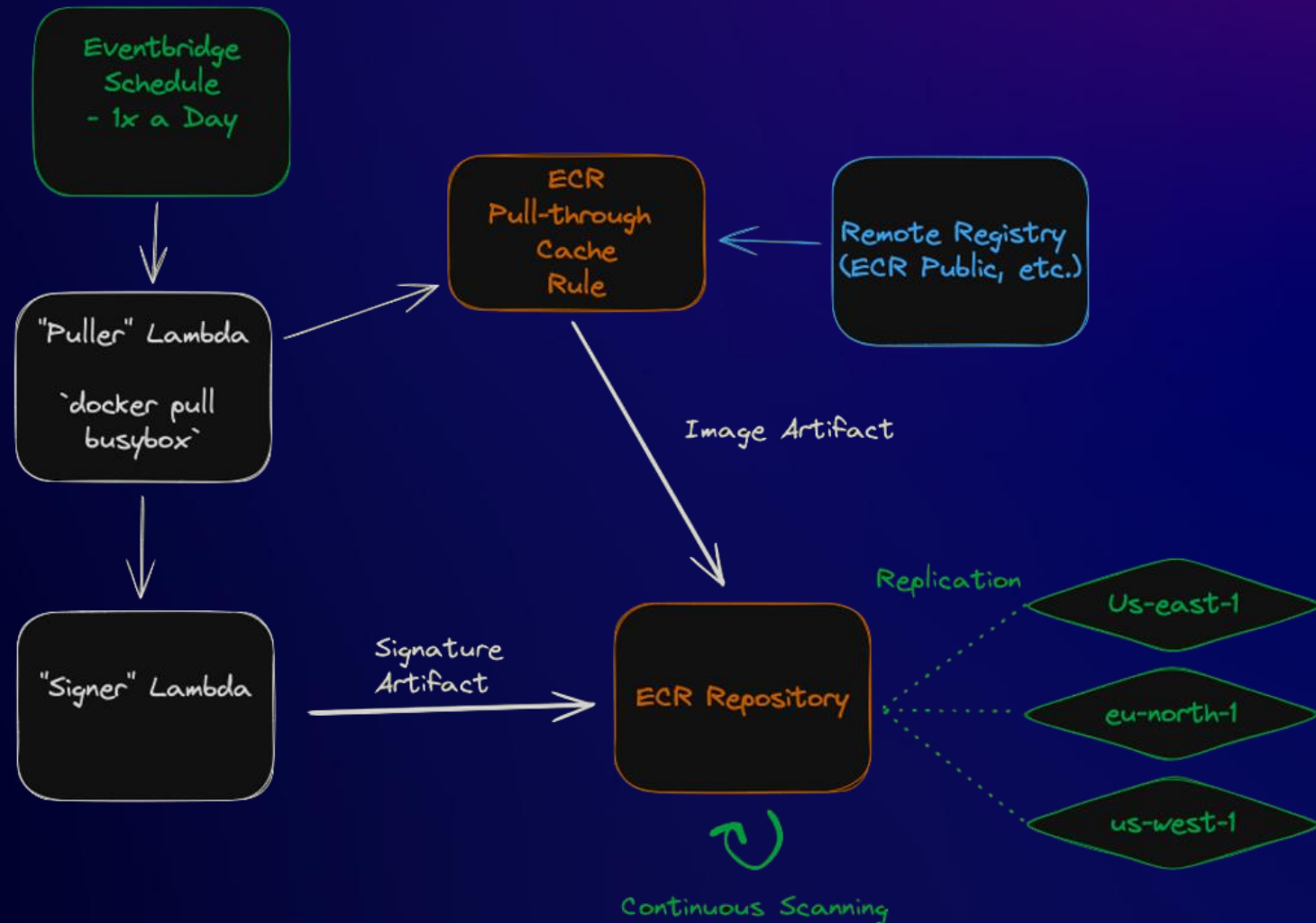
Quarantine new images	✓	AWS IAM with ECR repository creation templates
Once daily image procurement	✓	ECR pull through cache
Scan image content	✓	ECR enhanced scanning
Analyze vulnerability findings	✓	AWS Step Functions
Sign image and promote	✓	AWS Signer
Replicate images globally	✓	ECR replication

What does this mean in practice?

This solution can be built with ECR and AWS features

Amazon EventBridge Scheduler with AWS Step Functions drive the solution

Try it yourself!



Additional questions from the room

Thank you!

Matt Kang

X @kattmang

in linkedin.com/in/mattkangcodes

Adam Long

X @alpeople

in linkedin.com/in/adam-long



Please complete the session
survey in the mobile app



Github Repository
of the ECR ingestion solution

